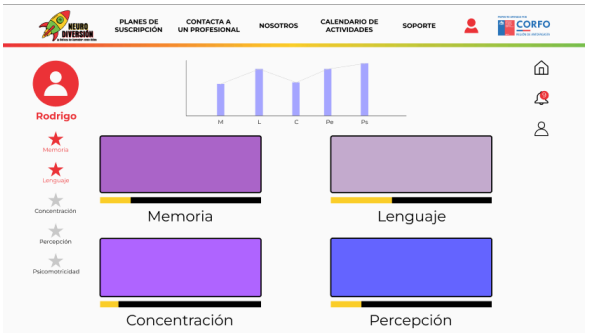
Milestone 2

The goal of Milestone 2 is to enable a responsible user to access an NNA user's profile, where they can view assigned activities, complete them, and record the results. This milestone focuses on the activity flow and progress visibility.

To access the nna_user view, a specific user must be selected:

1. **Principal nna_user IU:** This interface mainly allows access to the different activity categories and visualization of the nna_user's progress.



   a. **Information loading: Call the function 'get_nna_dashboard_overview'** using the id of the selected nna_user to get the student's information, permissions, metrics, view_mode and redirect_url (Only for certain cases that will be explained).

   b. **Example query:**

```
const loadDashboard = async (nnaId) => {
  const { data, error } = await supabase.rpc('get_nna_dashboard_overview', {
    p_nna_user_id: nnaId
});
  if (error) {
    console.error("Error loading dashboard:", error);
    return; // handle error
  }
  setStudent(data.student);
  setPermissions(data.permissions);
  setChartsData(data.progress_metrics);
```

};

c. **Data return:**

i. **‘student’ object:** For now, we will only use 'first_name' (The rest will be used when selecting the profile in the following milestones)

Field	Data type	Action in Frontend
id	UUID	Unique student identifier. Use as a key or reference.
first_name	String	First name. Show below the icon
last_name	String	It will be used in profile editing.
birthdate	Date (ISO)	It will be used in profile editing.
sex	String	It will be used in profile editing.
age_years	Integer	It will be used in profile editing.
plan_name	String	It will be used in profile editing.
has_active_plan	Boolean	It will be used in profile editing.

ii. **‘view_mode’**

view_mode value	Meaning of Business	Action in Frontend
'DASHBOARD_VIEW'	✅ Everything is fine. The child has already completed his diagnosis.	1. Redirect to the dashboard as usual. 2. Render the charts using progress_metrics. 3. Display the dashboard as normal.
'REDIRECT'	🛑 Blockage. The diagnosis is either nonexistent or incomplete.	1. Do not redirect to the dashboard normally. 2. Read redirect_url. 3. Execute router.push(data.redirect_url) immediately.

iii. **‘redirect_url’**

Field	Data type	Action in Frontend
redirect_url	String	**The Route.** If the mode is REDIRECT, here is the exact URL to which the user should be sent.

iv. **‘permissions’ object:**

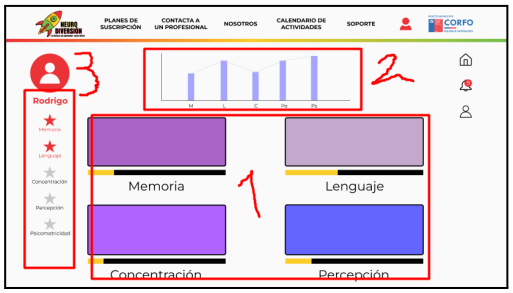
Field	Data type	Action in Frontend
can_edit_profile	Boolean	TRUE: Allows editing the 'name' in the profile (Future milestone) FALSE: Does not allow editing
can_view_activities	Boolean	TRUE: Allows access to categories: FALSE: Does not allow access to categories, **blocks the ability to click.**

v. **‘progress_metrics’ object:** Returns 1 row per section. If ‘view_mode’ is ‘REDIRECT’, this array will be empty (to avoid breaking components, but it should not be displayed). Its columns are:

Field	Data type	Description
section_name	String	Category name (e.g., "Memory").
dynamic_percentage	Float	Simple percentage of completeness. Use for performance charts.
section_progress_percentage	Float	Simple percentage of completeness

has_star	Boolean	**TRUE:** The score exceeds the goal.
total_activities	Integer	Total number of available activities (for debug)
activities_attempted	Integer	How many has the student tried? (for debug)

The data obtained in ‘progress_metrics’ is used in the following 3 main parts:



1. **Part 1: Activity categories:** To obtain the activity categories that must be displayed next to the progress bar for the nna_user in each of them, the following columns must be obtained from the view for each row:
    a. ‘section_name’
    b. ‘section_progress_percentage’

2. **Part 2: Summary Chart:** To get the percentage height of each bar in the chart, the following column must be obtained from the view for each row:
    a. ‘dynamic_percentage’

3. **Part 3: Completed category indicator**
    a. One star must be displayed for each category returned.
    b. To mark a colored star check the 'has_star' field

2. **Specific category view:** Accessed by selecting each of the categories in point 1 and displays all available activities that the user can perform.

a. **Information loading:** All necessary information is obtained llamando la funcion rpc '**get_nna_section_activities**', using the **nna_user** id and the selected **category id**:

i. **Input data:**

Parameter	Data type	Description
p_nna_user_id	String (UUID)	ID of the selected student.
p_section_id	Number	ID of the category/section to query

ii. **Use example:**

```
const { data, error } = await supabase.rpc('get_nna_section_activities',
{
        p_nna_user_id: nnaId,
        p_section_id: sectionId
});
return data;
```

iii. **Output data:** Activities list:

Parameter	Data type	Description / UI logic
activity_id	Number	IUnique identifier for the activity. **Use**: Serves as a key for mapping in React and as a parameter to navigate to the activity details.
is_success	Boolean | Null	Indicates the completion status of the activity: • true: Completed: It must be colored green and the letter "L" displayed • false/null: Otherwise, red and the letters "PL"

3. **Specific Activity View:** Accessed by selecting one of the activities from point 2, it displays the specific information of the selected activity with the possibility of marking the user's answer. It is composed of 4 parts:

a. **Information loading:** Use the rpc function 'get_nna_activity_detail' which obtains the complete content of a specific activity (video, instructions, pedagogical objective) and its completion status.

i. **Input data:**

Parameter	Data type	Description
p_nna_user_id	String (UUID)	ID of the selected student.
p_activity_id	Number	ID of the activity to consult.

ii. **Example use:**

```
const { data, error } = await supabase.rpc('get_nna_activity_detail', {
        p_nna_user_id: nnaId,
        p_activity_id: activityId
});
return data;
```

iii. **Output data:**

Parameter	Data type	Description / Frontend Usage
activity_id	Number	Activity ID (confirmation).
program_id	Number	Program ID it belongs to (For debug).
instruction	String	Instructional text for the parent/guardian.
video_url	String	Video URL. It can be null.
question	String	Main question
objetive_specific	String	Target Text
is_success	Boolean | Null	Current status of the activity: • true: Completed. • false: Not Completed. • null: Pending (Unanswered).
result_id	Number | Null	Row ID in nna_activity_result. Useful for debugging

iv. Part 1 (Paragraph): Get from: '**objetive_specific**'

v. Part 2 (Paragraph): Obtain from: '**instruction**'

vi. Part 3: Get from: '**question**'

vii. Part 4: Obtain from '**video_url**'

b. **Response buttons**: The activity must allow selecting one of the 2 buttons and marking it in green:

Observation: Load the button's state with 'is_success' (If it is not null)

c. **Confirmation button:** Call the function '**save_activity_result**' which saves or updates the result of an activity.

i. **Input data:**

Parameter	Data type	Description
p_nna_user_id	UUID (String)	The ID of the student/child who is doing the activity.
p_activity_id	Integer	The activity ID
p_result	Boolean	true = Logrado / Completeted Logrado false = Por lograr / Needs improvement Por lograr

ii. **Example use:**

```
const { data, error } = await supabase
        .rpc('save_activity_result', {
                p_nna_user_id: childId,
                p_activity_id: activityId,
                p_result: isSuccess
        });
```

d. **Completed Response Modal:** An informative modal should be displayed to the user depending on whether their answer ('**p_result**') was TRUE or FALSE:

If TRUE	If FALSE

i. Note: Only change the main text and the rest remains the same

ii. **"Continuar" (Continue) Button:**

It must redirect to the next activity. To return the ID of the activity to

load, the function "get_next_section_activity" must be called, using the ID of the nna_user, and the ID of the current activity:

```
const { data } = await supabase.rpc('get_next_section_activity', {

        p_nna_user_id: nnaId,
        p_current_activity_id: currentActivityId
}); return data;
```

iii. “Volver” (Return) Button:

1. It must redirect to the **principal nna_user UI** interface (Point 1)